

Caso: Agente de Análisis Conversacional con BigQuery y Cloud Run Functions

Módulo: Entornos híbridos: Dedicados y Cloud

Profesor: Anita Quevedo

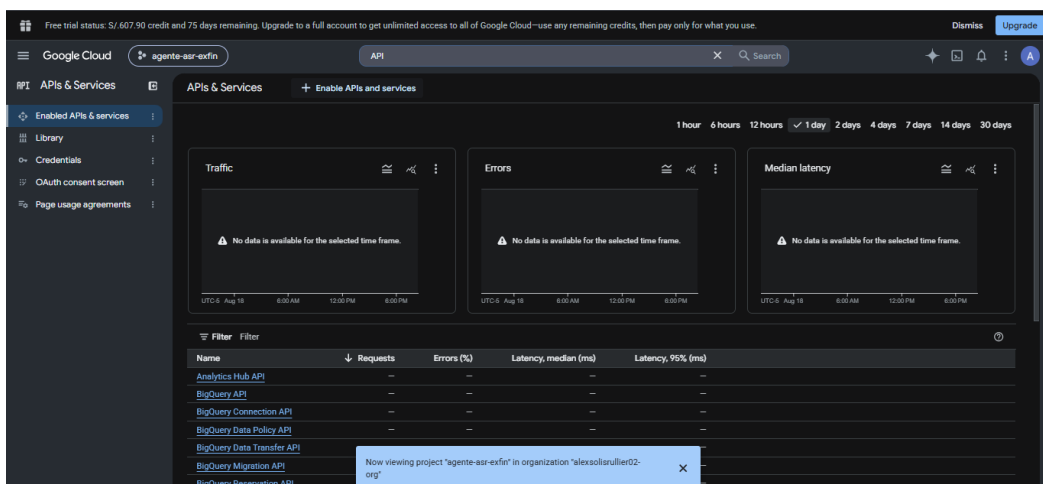
Estudiante: Alexander Arturo Solis Rullier

Objetivo: Diseñar, implementar y desplegar una solución web capaz de responder preguntas en lenguaje natural sobre datasets en **BigQuery**. Para ello, deberás configurar un **Data Agent**, integrar la **Conversational Analytics API** en una arquitectura de microservicios y exponer el servicio mediante una **Cloud Run Function** conectada a una interfaz web.

Fases del proyecto

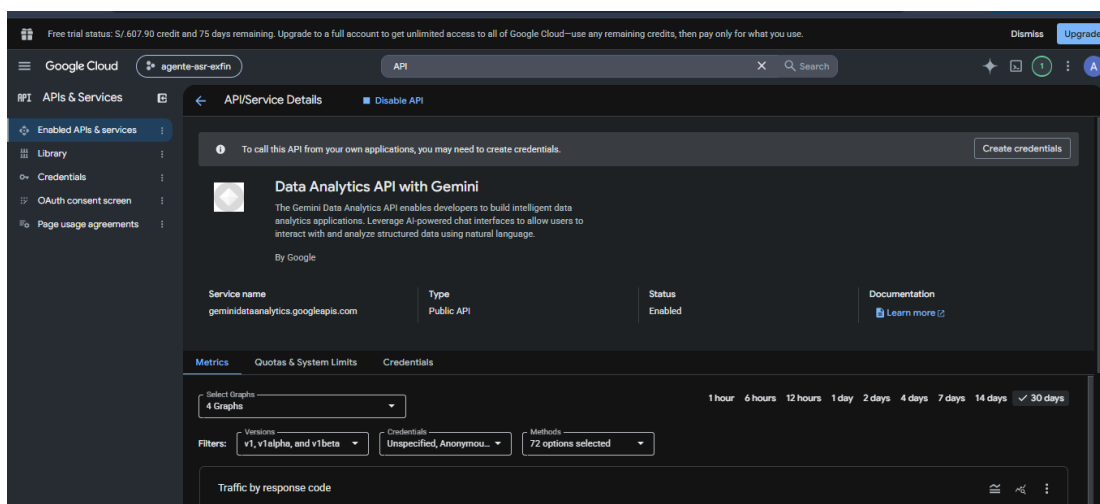
Fase 1: Configuración del Entorno y Agente en BigQuery

a. Creación del proyecto

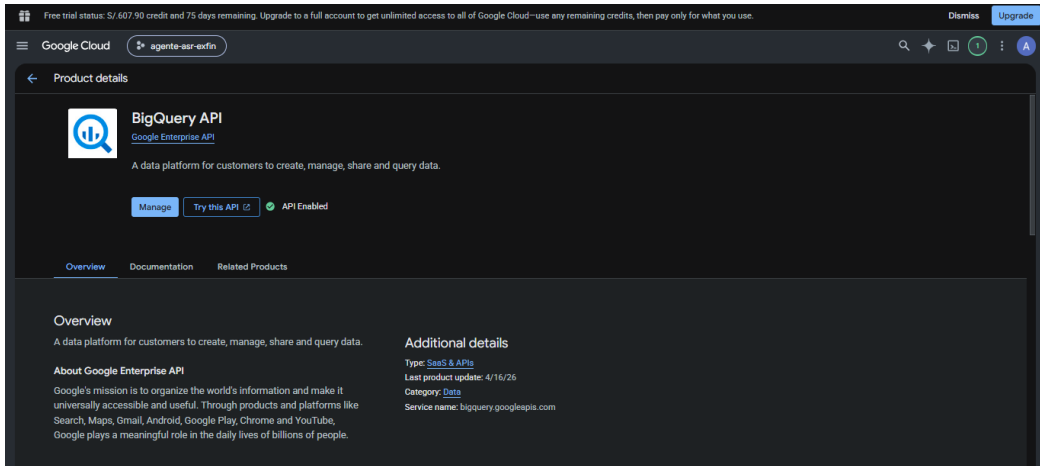


b. Habilitación de API's: APIs & Services → Library

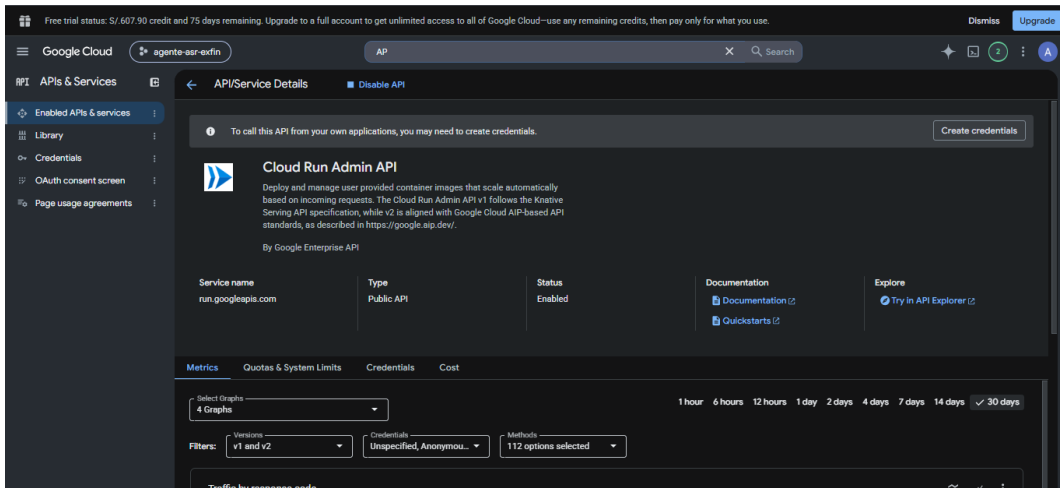
• Gemini Data Analytics API



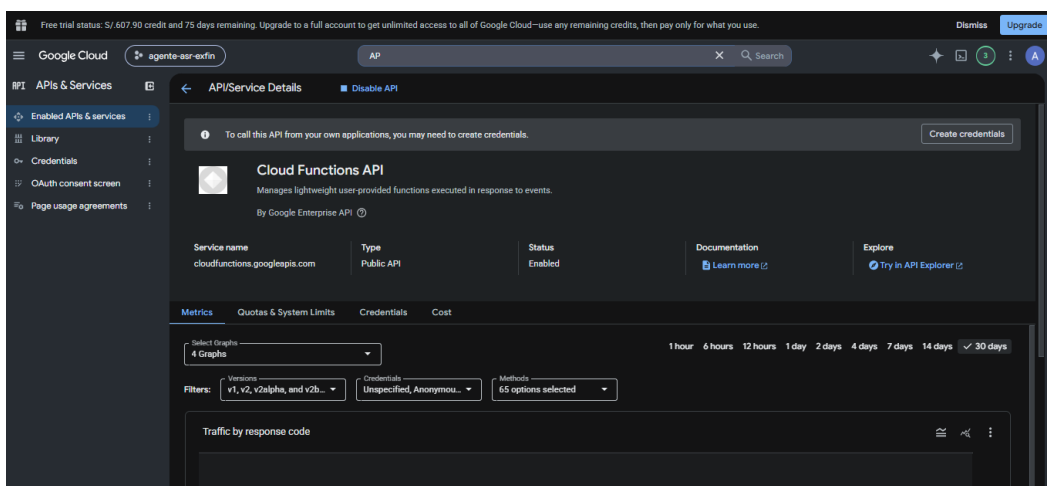
- **Big Query API**



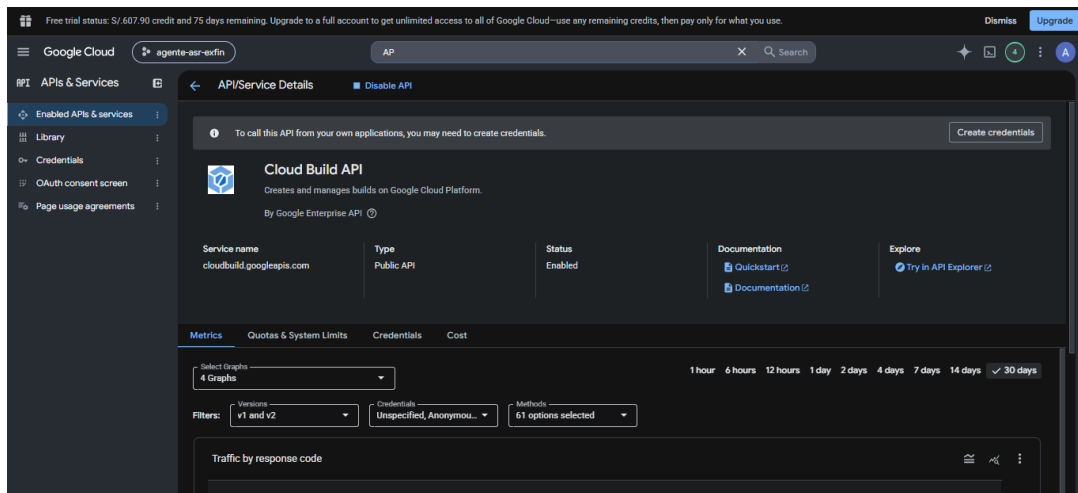
- **run.googleapis.com**



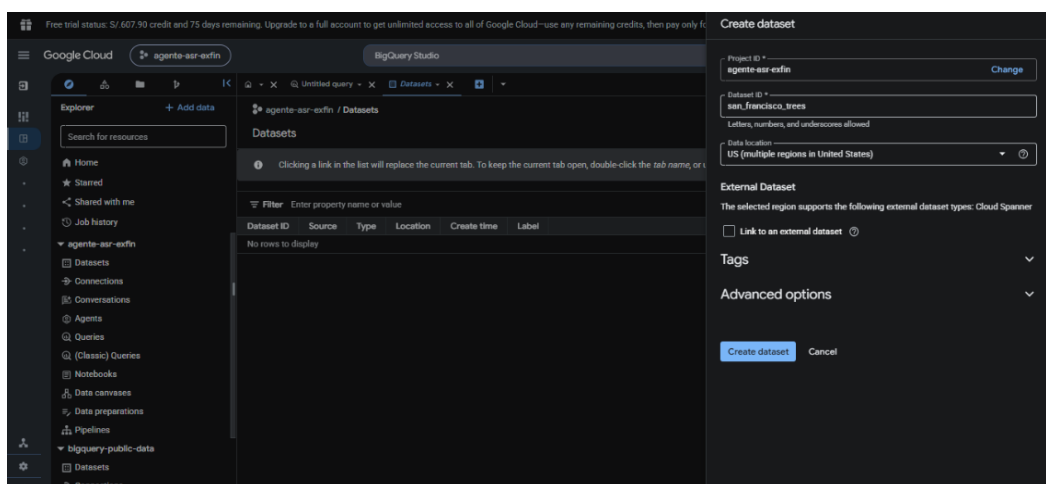
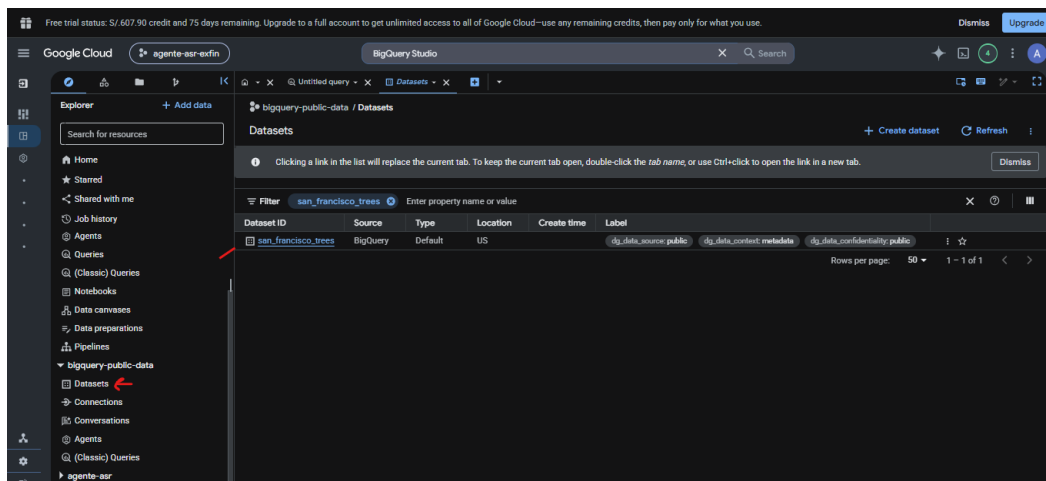
- **cloudfunctions.googleapis.com**

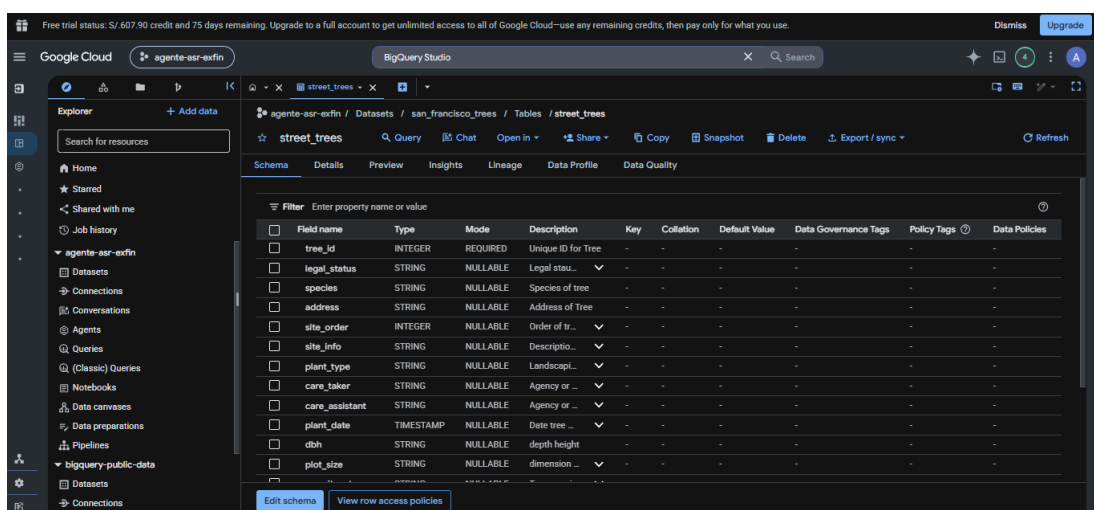
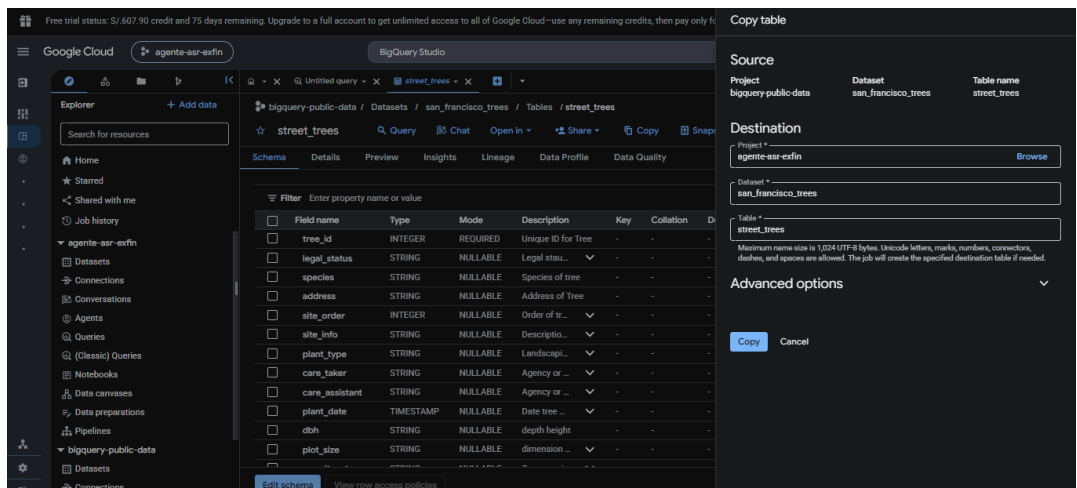


- **Creación del Data Agent**

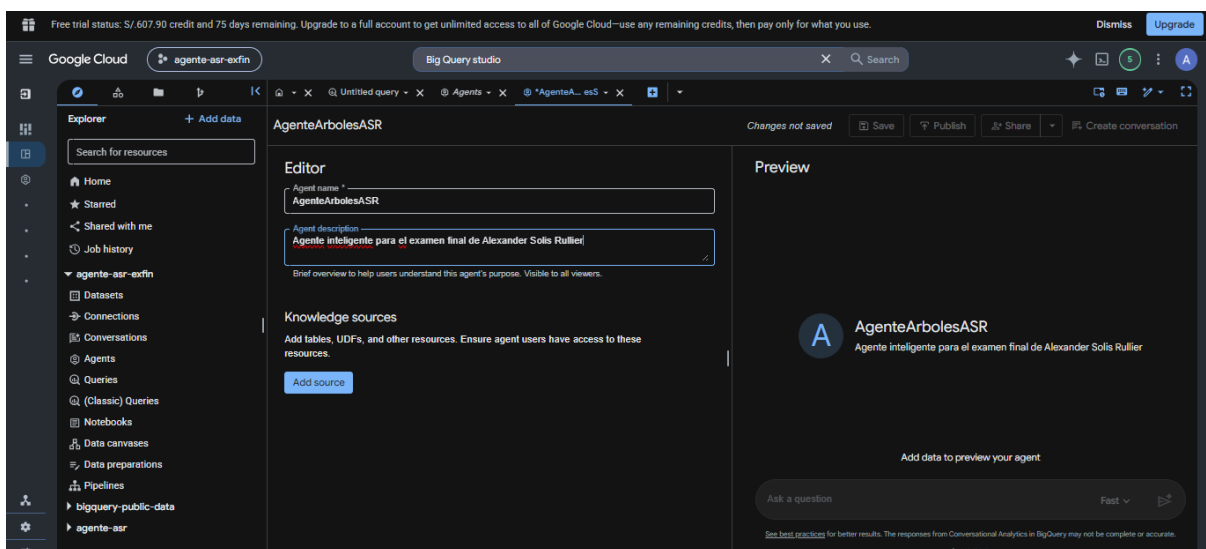


- Selecciona un dataset público: bigquery-public-data.san_francisco_trees





- **Nombre de agente:** AgenteArbolesASR

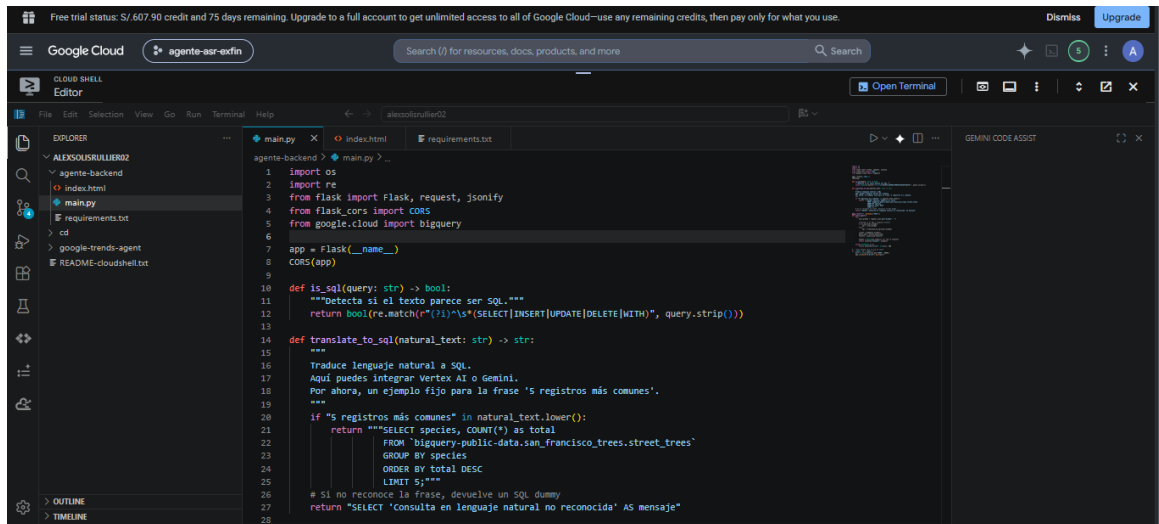


Fase 2: Desarrollo Backend (Cloud Run Function)

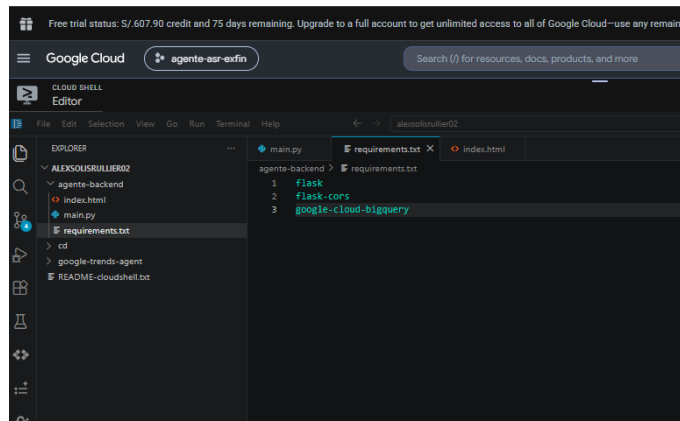
- Construcción de la API REST:

Construcción de la API REST:

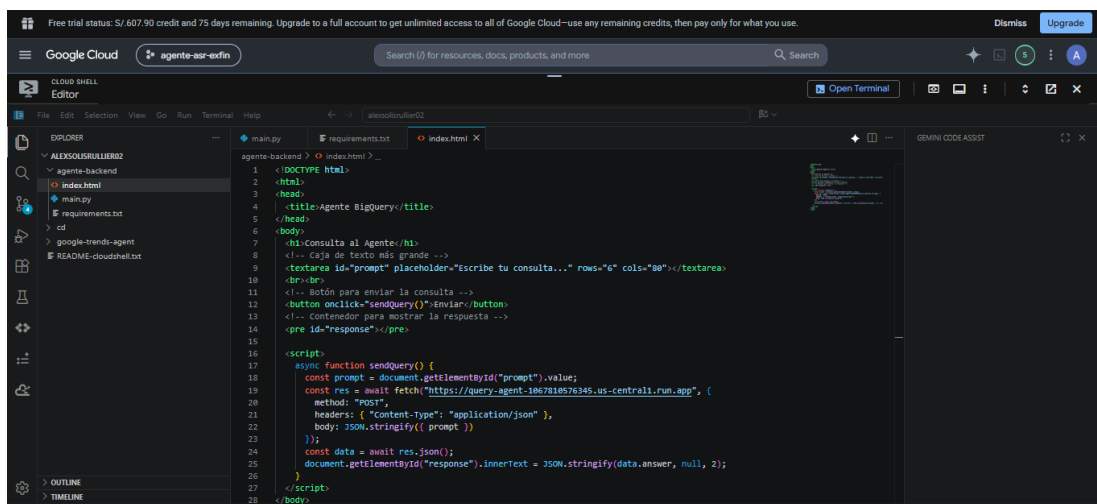
- Crea una función HTTP en Python (main.py y requirements.txt).
- Utiliza la librería cliente de Conversational Analytics (google-cloud-geminidataanalytics) para recibir un prompt enviado por el usuario y transmitirlo al Data Agent creado en la Fase 1.
- Maneja correctamente los encabezados CORS para permitir peticiones desde clientes web externos.



```
1 import os
2 import re
3 from flask import Flask, request, jsonify
4 from flask_cors import CORS
5 from google.cloud import bigquery
6
7 app = Flask(__name__)
8 CORS(app)
9
10 def is_sql(query: str) -> bool:
11     """Detecta si el texto parece ser SQL."""
12     return bool(re.match("(?i)-is-(SELECT|INSERT|UPDATE|DELETE|WITH)", query.strip()))
13
14 def translate_to_sql(natural_text: str) -> str:
15     """
16     Traduce lenguaje natural a SQL.
17     Aquí puedes integrar vertex AI o Gemini.
18     Por ahora, un ejemplo fijo para la frase '5 registros más comunes'.
19     """
20     if "5 registros más comunes" in natural_text.lower():
21         return """SELECT species, COUNT(*) as total
22 FROM `bigquery-public-data.san_francisco_trees.street_trees'
23 GROUP BY species
24 ORDER BY total DESC
25 LIMIT 5;"""
26     # Si no reconoce la frase, devuelve un SQL dummy
27     return "SELECT 'Consulta en lenguaje natural no reconocida' AS mensaje"
```

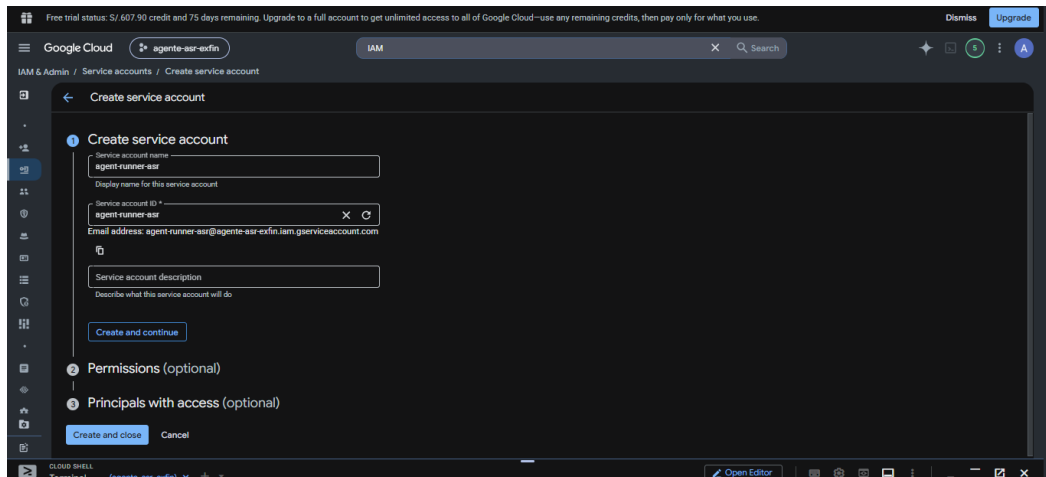


```
1 flask
2 flask-cors
3 google-cloud-bigquery
```



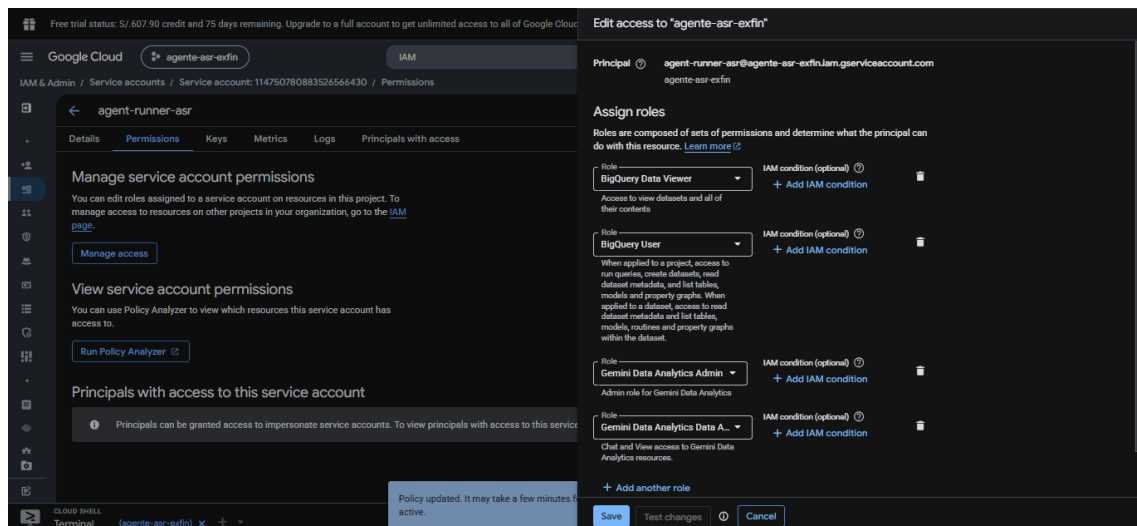
```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Agente BigQuery</title>
5 </head>
6 <body>
7 <h1>Consulta al Agente</h1>
8 <!-- Caja de texto más grande -->
9 <textarea id="prompt" placeholder="Escribe tu consulta..." rows="6" cols="80"></textarea>
10 <br>
11 <!-- Botón para enviar la consulta -->
12 <button onclick="sendQuery()">Enviar</button>
13 <!-- Contenedor para mostrar la respuesta -->
14 <pre id="response"></pre>
15
16 <script>
17 async function sendQuery() {
18     const prompt = document.getElementById("prompt").value;
19     const res = await fetch("https://query-agent-1867816576345-us-central1.run.app", {
20         method: "POST",
21         headers: { "Content-Type": "application/json" },
22         body: JSON.stringify({ prompt })
23     });
24     const data = await res.json();
25     document.getElementById("response").innerText = JSON.stringify(data.answer, null, 2);
26 }
27
28 </script>
29 </body>
```

- **Creación de la cuenta de servicio: Seguridad e Identidad (IAM):**



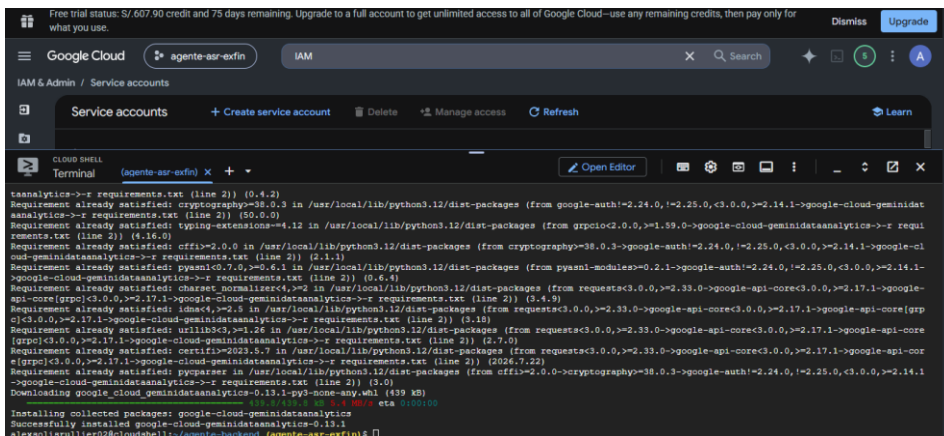
- **Asignación de roles para la cuenta de servicio**

Configura la Service Account utilizada por la función con los roles estrictamente necesarios (BigQuery Data Viewer, BigQuery User y permisos de acceso a Gemini Data Analytics).



Fase 3: Despliegue en GCP

Despliega la función utilizando gcloud functions deploy (versión 2nd Gen, runtime Python 3.11+).



```
Free trial status: $/ 607.90 credit and 75 days remaining. Upgrade to a full account to get unlimited access to all of Google Cloud—use any remaining credits, then pay only for what you use. Dismiss Upgrade

Google Cloud agente-asr-exfin IAM Search

IAM & Admin / Service accounts

Service accounts + Create service account Delete Manage access Refresh Learn

CLOUD SHELL Terminal (agente-asr-exfin) x + Open Editor

alexsolisrullier02@cloudshell:~/agente-backend (agente-asr-exfin)$ python3 -c "from google.cloud import geminidataanalytics; print('OK!')"
OK
alexsolisrullier02@cloudshell:~/agente-backend (agente-asr-exfin)$ python3 main.py
* Serving Flask app 'main'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://10.48.0.3:8080
Press CTRL+C to quit
^Calexsolisrullier02@cloudshell:~/agente-backend (agente-asr-exfin)$
```

```
Free trial status: $/ 607.90 credit and 75 days remaining. Upgrade to a full account to get unlimited access to all of Google Cloud—use any remaining credits, then pay only for what you use. Dismiss Upgrade

Google Cloud agente-asr-exfin Search (/) for resources, docs, products, and more Search

CLOUD SHELL Terminal (agente-asr-exfin) x (agente-asr-exfin) x + Open Editor

alexsolisrullier02@cloudshell:~/agente-backend (agente-asr-exfin)$ gcloud run deploy query-agent \
--source . \
--region=us-central1 \
--allow-unauthenticated \
--service-account=agent-runner-asr@agente-asr-exfin.iam.gserviceaccount.com
Deploying from source requires an Artifact Registry Docker repository to store built containers. A repository named [cloud-run-source-deploy] in region [us-central1] will be created.
Do you want to continue (Y/n)? Y
Building using Buildpacks and deploying container to Cloud Run service [query-agent] in project [agente-asr-exfin] region [us-central1]
Building and deploying new service...
Validating configuration...done
Creating Container Repository...done
Uploading sources...done
Building Container... Logs are available at [ https://console.cloud.google.com/cloud-build/builds;region=us-central1/26cd2e08-c07d-46ac-ac49-71977a436149?project=1067810576345 ]...done
Setting IAM Policy...done
Creating Revision...done
Routing traffic...done
Done.
Service [query-agent] revision [query-agent-00001-wcn] has been deployed and is serving 100 percent of traffic.
Service URL: https://query-agent-1067810576345.us-central1.run.app
alexsolisrullier02@cloudshell:~/agente-backend (agente-asr-exfin)$ "C
alexsolisrullier02@cloudshell:~/agente-backend (agente-asr-exfin)$ curl -X POST https://query-agent-1067810576345.us-central1.run.app \
-H "Content-Type: application/json" \
-d '{"prompt": "SELECT species, COUNT(*) as total FROM `bigquery-public-data.san_francisco_trees.street_trees` GROUP BY species ORDER BY total DESC LIMIT 5;"}'
{"answer": [{"species": "Platanus x hispanica", "total": 11953}, {"species": "Tree (s)", "total": 11818}, {"species": "Lophostemon confertus", "total": 11808}, {"species": "Metrosideros excelsa", "total": 8912}, {"species": "Tristanopsis laurina", "total": 7532}]}
alexsolisrullier02@cloudshell:~/agente-backend (agente-asr-exfin)$
```

- **Obtén y verifica la URL pública asignada al endpoint HTTP.**

cd ~/agente-backend

URL: <https://query-agent-1067810576345.us-central1.run.app>

- **Comando correcto para Cloud Run**

gcloud run deploy query-agent \

--source . \

--region=us-central1 \

--allow-unauthenticated \

[--service-account=agent-runner-asr@agente-asr-exfin.iam.gserviceaccount.com](#)

- **Probar el endpoint con curl**

curl -X POST https://query-agent-1067810576345.us-central1.run.app \

-H "Content-Type: application/json" \

-d '{"prompt": "SELECT species, COUNT(*) as total FROM `bigquery-public-data.san\_francisco\_trees.street\_trees` GROUP BY species ORDER BY total DESC LIMIT 5;"}'

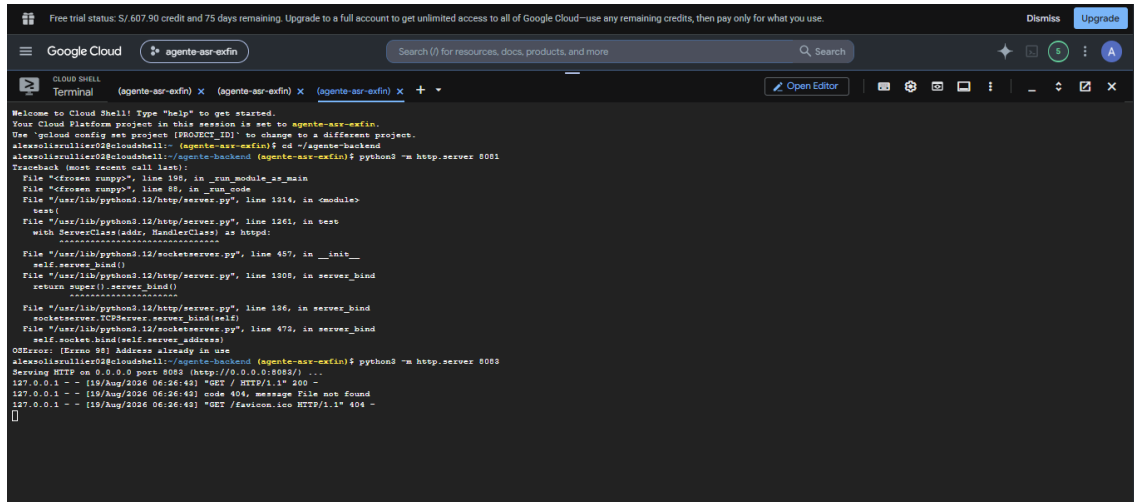
Fase 4: Interfaz Web y Pruebas

a. Crear el archivo index.html

Diseña una interfaz web simple (`index.html` con HTML/JavaScript) que contenga:

- Un campo de texto para ingresar consultas en lenguaje natural.
- Un botón de envío que consuma la URL de tu Cloud Run Function vía `fetch` o `axios`.
- Un contenedor para visualizar la respuesta entregada por el agente.

- `cd ~/agente-backend`
- `python3 -m http.server 8083`



```
Free trial status: $/ 607.90 credit and 75 days remaining. Upgrade to a full account to get unlimited access to all of Google Cloud—use any remaining credits, then pay only for what you use. Dismiss Upgrade

Google Cloud agente-asr-efin Search (/) for resources, docs, products, and more

CLOUD SHELL Terminal (agente-asr-efin) x (agente-asr-efin) x (agente-asr-efin) x + Open Editor

Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to agente-asr-efin.
Use "gcloud config set project [PROJECT_ID]" to change to a different project.
alexcollins@cloudshell:~ (agente-asr-efin) $ cd ~/agente-backend
alexcollins@cloudshell:~/agente-backend (agente-asr-efin) $ python3 -m http.server 8083
Traceback (most recent call last):
  File "<frozen runpy>", line 196, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "/usr/lib/python3.12/http/server.py", line 1214, in __module__
    test()
  File "/usr/lib/python3.12/http/server.py", line 1261, in test
    with ServerClass(addr, HandlerClass) as httpd:
    ~~~~~^~~~~~
  File "/usr/lib/python3.12/socketserver.py", line 457, in __init__
    self.server_bind()
  File "/usr/lib/python3.12/http/server.py", line 1908, in server_bind
    return super().server_bind()
    ~~~~~^~~~~~
  File "/usr/lib/python3.12/http/server.py", line 196, in server_bind
    socketserver.TCPServer.server_bind(self)
  File "/usr/lib/python3.12/socketserver.py", line 470, in server_bind
    self.socket.bind(self.server_address)
OSError: [Errno 98] Address already in use
alexcollins@cloudshell:~/agente-backend (agente-asr-efin) $ python3 -m http.server 8083
Serving HTTP on 0.0.0.0 port 8083 (http://0.0.0.0:8083/) ...
127.0.0.1 - - [19/Aug/2024 06:26:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [19/Aug/2024 06:26:42] code 404, message File not found
127.0.0.1 - - [19/Aug/2024 06:26:43] "GET /favicon.ico HTTP/1.1" 404 -
```

- b. Realiza pruebas de consulta (ej. "¿Cuáles son los 5 registros más comunes en el dataset?").



8084-cs-799888913098-default.cs-us-east1-pkhd.cloudshell.dev

Consulta al Agente

¿Cuáles son los 5 registros más comunes del dataset?

Enviar

```
[
  {
    "species": "Platanus x hispanica :: Sycamore: London Plane",
    "total": 11953
  },
  {
    "species": "Tree(s) ::",
    "total": 11818
  },
  {
    "species": "Lophostemon confertus :: Brisbane Box",
    "total": 10309
  },
  {
    "species": "Metrosideros excelsa :: New Zealand Xmas Tree",
    "total": 8912
  },
  {
    "species": "Tristaniaopsis laurina :: Swamp Myrtle",
    "total": 7532
  }
]
```


8084-cs-799888913098-default.cs-us-east1-pkhd.cloudshell.dev

Consulta al Agente

¿Cuántas son las especies únicas del dataset?

Enviar

```
[
  {
    "total_especies": 587
  }
]
```

8084-cs-799888913098-default.cs-us-east1-pkhd.cloudshell.dev

Consulta al Agente

¿Cuál es el registro más común del dataset?

Enviar

```
[
  {
    "species": "Platanus x hispanica :: Sycamore: London Plane",
    "total": 11953
  }
]
```